

Data frames: a data structure for statistical analysis

Matteo Fasiolo

Creating a Scoring Table

Imagine we wanted to create a “table” which stores data of different types:

student number,	name,	staff,	score
1	teo,	T,	70
2	jack,	T,	80
3	emma,	F,	70
...			

We cannot use `matrix` as it only allows single type data.

Lists can be used to store data of different types, can we use that?

```
stu_no <- c(1L, 2L, 3L)
name <- c("teo", "jack", "emma")
staff <- c(T, T, F)
score <- c(70, 80, 70)
```

```
score_table <- list(no = stu_no, name = name,
                    staff = staff, score = score)
```

This works:

```
score_table$name
```

```
[1] "teo" "jack" "emma"
```

```
score_table[[2]]
```

```
[1] "teo" "jack" "emma"
```

```
score_table$name[1]
```

```
[1] "teo"
```

But recall the data

student number	name	staff	score
1	teo,	T,	70
2	jack,	T,	80
3	emma,	F,	70
...			

To extract all information on Emma we must do:

```
score_table[[1]][3]
```

```
[1] 3
```

```
score_table[[2]][3]
```

```
[1] "emma"
```

```
score_table[[3]][3]
```

```
[1] FALSE
```

(Exercise: can you do this more compactly with `lapply?`)

Data Frames

Point is that list does not directly represent tabular format.

A data.frame does:

```
score_table <- data.frame(stu_no, name, staff, score)
```

```
score_table
```

	stu_no	name	staff	score
1	1	teo	TRUE	70
2	2	jack	TRUE	80
3	3	emma	FALSE	70

Extract info about Emma:

```
score_table[3, ]
```

	stu_no	name	staff	score
3	3	emma	FALSE	70

So data.frame can be accesses like a matrix:

```
score_table[3, 1]
```

```
[1] 3
```

```
score_table[1:2, ]
```

	stu_no	name	staff	score
1	1	teo	TRUE	70
2	2	jack	TRUE	80

We can also perform numeric operations on numeric columns:

```
score_table[, c(1, 4)] * 2 # Try score_table * 2
```

	stu_no	score
1	2	140
2	4	160
3	6	140

Can also access a data.frame like a list:

```
score_table$name
```

```
[1] "teo" "jack" "emma"
```

```
score_table[[2]]
```

```
[1] "teo" "jack" "emma"
```

```
score_table[c(1, 2)]
```

```
  stu_no name
1      1  teo
2      2 jack
3      3 emma
```

If can also mix matrix-like and list-like indexing:

```
score_table[score_table$name == "emma", ]
```

```
  stu_no name staff score
3      3 emma FALSE    70
```

Using list-like access we add/remove columns (variables):

```
score_table$age <- c(23, 23, 21)
```

```
score_table
```

	stu_no	name	staff	score	age
1	1	teo	TRUE	70	23
2	2	jack	TRUE	80	23
3	3	emma	FALSE	70	21

```
score_table$age <- NULL
```

Using matrix-like access we can remove or add:

```
( score_table <- score_table[-1, ] )
```

	stu_no	name	staff	score
2	2	jack	TRUE	80
3	3	emma	FALSE	70

```
score_table[3, ] <- list(1, "Jacob", TRUE, 60) #Try c()
```


Summarizing Data Frame

You can also get a summary of your data frame:

```
summary(score_table)
```

stu_no	name	staff
Min. :1.0	Length:3	Mode :logical
1st Qu.:1.5	Class :character	FALSE:1
Median :2.0	Mode :character	TRUE :2
Mean :2.0		
3rd Qu.:2.5		
Max. :3.0		

data.frames are a basic input for many modelling functions: `lm`, `glm`, `gam`, ...

Case Study: Predicting MNIST Digits.

We use the MNIST data set of handwritten digits.

It contains images stored as matrices of size (28×28) .

Structure of the data:

- ▶ Images: $X \in \mathbb{R}^{60000 \times 784}$ ($\sqrt{784} = 28$)
- ▶ Labels: $Y \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 0\}^{60000 \times 1}$
- ▶ Testing Images: $\tilde{X} \in \mathbb{R}^{10000 \times 784}$
- ▶ Testing Labels: $\tilde{Y} \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 0\}^{10000 \times 1}$

We can directly use these matrices within our program.

But using data frames can be tidier.

GOAL: predict the labels (digits) in the test set.

This is a **classification** problem.

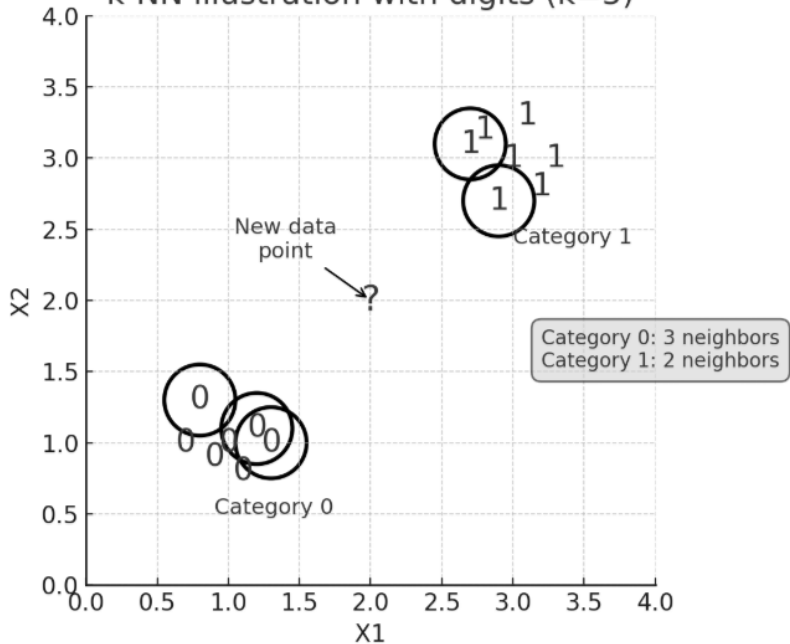
Given an image in the test set, we predict which digit it represents.

We assume can not see labels in the test set.

We will use a simple k-nearest neighbour (k-NN) algorithm:

1. For a test image, find the k training images which are closest.
 - ▶ Distance is Euclidean distance between vectors in $\mathbb{R}^{784}$.
2. Predict the label of the test image as the most common label among its k nearest neighbours.

k-NN illustration with digits (k=5)



Let us download the data

```
library(dslabs)
# Need internet access!
# Do save(file = "mnist.RData", mnist)
# Then load("mnist.RData")
mnist <- read_mnist()
```

```
str(mnist)
```

List of 2

\$ train:List of 2

..\$ images: int [1:60000, 1:784] 0 0 0 0 0 0 0 0 0 0 0 ...

..\$ labels: int [1:60000] 5 0 4 1 9 2 1 3 1 4 ...

\$ test :List of 2

..\$ images: int [1:10000, 1:784] 0 0 0 0 0 0 0 0 0 0 0 ...

..\$ labels: int [1:10000] 7 2 1 0 4 1 4 9 5 9 ...

Note: str shows structure of R objects.

We rearrange the data and subsample:

```
sub <- 1:1000
```

```
train <- list("images" = mnist$train$images[sub, ],  
             "labels"  = mnist$train$labels[sub],  
             "istest"  = rep(FALSE, length(sub)))
```

where

```
rep(FALSE, 3)
```

```
[1] FALSE FALSE FALSE
```

```
test <- list("images" = mnist$test$images[sub, ],  
            "labels"  = mnist$test$labels[sub],  
            "istest"  = rep(TRUE, length(sub)))
```

```
dat_mnist <- data.frame(  
  "images" = I( rbind(train$images, test$images) ),  
  "labels" = c(train$labels, test$labels),  
  "istest" = c(train$istest, test$istest))
```

```
dim(dat_mnist)
```

```
[1] 2000    3
```

```
dim(dat_mnist$images)
```

```
[1] 2000 784
```

```
str(dat_mnist)
```

```
'data.frame':    2000 obs. of  3 variables:  
 $ images: 'AsIs' int [1:2000, 1:784] 0 0 0 0 0 0 0 0 0 0 0  
 $ labels: int    5 0 4 1 9 2 1 3 1 4 ...  
 $ istest: logi   FALSE FALSE FALSE FALSE FALSE FALSE ...
```

The `I()` says images matrix should be stored in the data frame **as is**.

Without using `I()`, R breaks the images matrix into its columns:

```
dat_wrong <- data.frame(  
  "images" = rbind(train$images, test$images),  
  "labels" = c(train$labels, test$labels),  
  "istest" = c(train$istest, test$istest))
```

```
dim(dat_wrong)
```

```
[1] 2000  786
```


Now each row identifies a digit, we want to visualise them:

```
library(imager)
plot_im <- function(x, lab = ""){
  img <- matrix(x, nrow = sqrt(length(x)))
  plot(as.cimg(img), axes = FALSE)
  title(lab)
}
```

```
plot_im(dat_mnist$images[1, ], lab = dat_mnist$labels[1])
```

5



Find Neighbours

Function that finds the indices of 20 nearest neighbours of image x:

```
find_nei <- function(x, dat){  
  train <- dat[dat$istest == FALSE, ]  
  n <- nrow(train$images)  
  dist <- numeric(n)  
  for(i in 1:n){ # Exercise: can you avoid the loop?  
    dist[i] <- sqrt(sum((x-train$images[i, ])^2))  
  }  
  return(order(dist)[1:20])  
}
```

Where note that

```
order(c(10, 0.1, 5.2))
```

```
[1] 2 3 1
```

Test our function

```
kk <- 1002 # 2nd digit in test set
```

```
nei <- find_nei(dat_mnist$images[kk, ], dat = dat_mnist)
```

```
nei
```

```
[1] 361 77 603 51 733 275 547 841 379 831 675 263 631 71  
[20] 553
```

```
dat_mnist$labels[nei]
```

```
[1] 2 2 8 3 2 6 3 3 2 2 2 2 6 1 6 3 1 6 2 1
```

We use the neighbours labels to make predictions.

Make predictions

```
dat_mnist$labels[nei]
```

```
[1] 2 2 8 3 2 6 3 3 2 2 2 2 6 1 6 3 1 6 2 1
```

Useful command here is:

```
table(c("c", "b", "c", "c", "b", "a"))
```

```
a b c
```

```
1 2 3
```

```
( tab <- table(dat_mnist$labels[nei]) )
```

```
1 2 3 6 8
```

```
3 8 4 4 1
```

```
names(tab)
```

```
[1] "1" "2" "3" "6" "8"
```

```
names(tab)[ tab == max(tab) ]
```

```
[1] "2"
```

```
plot_im(dat_mnist$images[kk, ],  
        lab = dat_mnist$labels[kk])
```

2



Wrapping the prediction step

So our prediction function is:

```
predict_lab <- function(x, dat){  
  
  nei <- find_nei(x, dat)  
  
  nei_labs <- dat$labels[nei]  
  
  tab <- table(nei_labs)  
  
  pred <- names(tab)[tab == max(tab)][1] # [1] for ties  
  
  pred <- as.integer(pred)  
  
  return(pred)  
  
}
```

Check if it works:

```
predict_lab(dat_mnist$images[kk, ], dat = dat_mnist)
```

```
[1] 2
```

Get predicted labels for all images and store in data.frame:

```
dat_mnist$preds <- apply(dat_mnist$images,  
                          1,  
                          predict_lab,  
                          dat = dat_mnist) # ...
```

This takes a while...

Add error column:

```
dat_mnist$error <- dat_mnist$preds - dat_mnist$labels
```

```
str(dat_mnist)
```

```
'data.frame':  2000 obs. of  5 variables:
 $ images: 'AsIs' int  [1:2000, 1:784] 0 0 0 0 0 0 0 0 0 0 0
 $ labels: int    5 0 4 1 9 2 1 3 1 4 ...
 $ istest: logi   FALSE FALSE FALSE FALSE FALSE FALSE ...
 $ preds : int    5 0 4 1 9 2 1 3 1 4 ...
 $ error : int    0 0 0 0 0 0 0 0 0 0 0 ...
```

```
train_err <- dat_mnist$error[dat_mnist$istest == FALSE]
test_err  <- dat_mnist$error[dat_mnist$istest == TRUE]
```



```
table(train_err)
```

```
train_err
```

-9	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5
2	2	12	13	13	2	13	24	20	849	7	8	3	5	19

```
table(test_err)
```

```
test_err
```

-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6
7	8	14	15	9	9	39	48	765	12	14	1	10	41	7

Mean absolute errors:

```
mean( abs( train_err ) )
```

```
[1] 0.575
```

```
mean( abs( test_err ) )
```

```
[1] 0.799
```

We do better on the train than on the testing set, why?

Order of complexity

How many computations does k-NN in terms of:

- number of images in training set (n)
- number of images in test set (n_T)
- size of the images ($p \times p$)?

Euclidean distance between two images:

$$\text{dist}(x, y) = \left(\sum_{j=1}^{p^2} (x_j - y_j)^2 \right)^{1/2}$$

so p^2 subtractions, p^2 squares, $p^2 - 1$ additions, 1 sqrt $\rightarrow O(p^2)$.

For a test image, compute distance to all training images: $O(n p^2)$.

So total cost is $O(n_T n p^2)$.

Conclusion

A `data.frame` combines data of different types together, in a table:

- ▶ Datasets are organized as “observations” and “columns (variables)”.
- ▶ a `data.frame` can be accessed like a matrix or like a list.

A `data.frame` provides a clean and unified way of managing and accessing datasets in our program.

- ▶ Instead of managing several vectors, we only need to manage one data frame for each dataset.

Not every application/data set can be organised in data frames, but many can!